

SYSTEM ARCHITECTURE AND TECHNICAL SPECIFICATION

BRAINLYX-LMS

Bui Cong Thiet & Colleagues

May 2026

1 Introduction

BrainlyX is a modern, cloud-based Learning Management System (LMS) designed to cater to the educational dynamics of high schools, colleges, and universities. The platform delivers a lightweight environment integrated with an intuitive user interface, exceptional user experience (UX), and high-standard data encryption.

The system is developed by Bui Cong Thiet along with research partners at Ho Chi Minh City Open University and various collaborative institutions, aiming to bridge the gap between traditional administrative workflows and seamless digital education.

2 System Architecture

BrainlyX-LMS is engineered based on a decoupled, multi-tier micro-services-ready architecture. The separation between the Presentation Layer, Application Logic Layer, and Data Infrastructure Layer ensures high availability, system maintainability, and horizontal scalability.

The structural blueprint of the system is elaborated in the sections below:

2.1 Presentation Layer (Frontend System)

The Presentation Layer focuses entirely on rendering fluid interfaces, executing client-side state management, and optimizing the End-User Experience (UX) across heterogeneous devices.

- **Framework Core:** Built using **React.js** (Virtual DOM approach), enabling single-page application (SPA) behaviors that eliminate jarring page reloads during learning sessions.
- **Styling Architecture:** Implemented via **Tailwind CSS**, utilizing atomic, utility-first CSS design patterns. This minimizes the production CSS bundle size and enforces strict design tokens (spacing, typography, responsive breakpoints).
- **Client-Side Routing & State Management:** Managed through React Router DOM for asynchronous route transitions, and Context API / Redux Toolkit for caching user profiles and institutional configuration states.
- **Deployment Network:** Deployed on **Vercel's Serverless Edge Network**. Front-end assets are built statically via optimized build-pipelines and distributed across global Content Delivery Networks (CDNs), decreasing Time to First Byte (TTFB).

2.1.1 User Authentication Interfaces

The application isolates entry-point views to prevent unauthorized layout shifting:

- **Native Portal:** Interactive credentials input with instantaneous Regex validation (e.g., verifying student email formats) and modern security UX implementations (Anti-Brute-force feedback, CAPTCHA anchors).

- **Federated Identity Gateway:** A unified interface incorporating Single Sign-On (SSO) buttons, customized specifically for instantaneous institutional Google profile synchronization.

2.2 Application and Business Logic Layer (Backend System)

Serving as the computational engine of BrainlyX-LMS, this layer encapsulates the educational business rules, course workflow engines, and real-time computation mechanisms.

- **Runtime Environment:** Powered by **Node.js**, utilizing its asynchronous, non-blocking I/O model driven by an event loop. This enables the server to handle thousands of concurrent API requests from students during peak exam periods without thread starvation.
- **Web Framework:** Implemented with Express.js to construct a structured RESTful API gateway, parsing client incoming payloads and formatting standard JSON responses.
- **Modular Service Architecture:** Code logic is cleanly separated into distinct design layers:
 - *Routes Layer:* Intercepts URLs and routes requests to appropriate controllers.
 - *Controllers Layer:* Manages HTTP request-response flows and payload structural validation.
 - *Services Layer:* Houses core educational logic (e.g., calculating grade averages, checking assignment deadlines).

2.3 Authentication and Authorization Layer (Security Core)

Security within BrainlyX-LMS is deeply embedded across communication protocols and identity control matrices.

2.3.1 Identity Verification System

The platform operates on a dual-authentication mechanism:

- **Internal Authentication Engine:** Uses **bcrypt** with a high work factor salt (`saltRounds = 12`) for cryptographic hashing of passwords. The system issues stateless, cryptographically signed **JSON Web Tokens (JWT)** containing user claims upon verification. Tokens are divided into short-lived Access Tokens (stored in-memory) and long-lived Refresh Tokens (stored in HTTP-only, SameSite cookies to mitigate Cross-Site Scripting - XSS, and Cross-Site Request Forgery - CSRF attacks).
- **External Identity Federation (Google OAuth 2.0):** Leverages Google APIs to construct a secure handshake protocol. The backend intercepts the Authorization Code, securely exchanges it for an ID Token from Google Authorization Servers, decodes the institutional profile, and provisions local system sessions automatically.

2.3.2 Role-Based Access Control (RBAC)

To satisfy university governance, a rigid RBAC middleware is injected into the request pipeline. Endpoints are guarded by hierarchical permission checks:

1. **Administrator:** Absolute system access, managing institutional master data, department hierarchies, and global system logs.
2. **Instructor:** Rights restricted to creating course spaces, publishing syllabi, setting grading rubrics, and evaluating submitted assignments.
3. **Student:** Permissions limited to enrolling in assigned courses, downloading learning materials, participating in discussion boards, and submitting tasks.

2.4 Data Management Layer (Storage Structuring)

Persistence and integrity of transactional data are sustained by a hardened relational infrastructure.

- **Database Management System (DBMS):** Driven by **MySQL** Production Engine. The relational paradigm guarantees ACID properties (Atomicity, Consistency, Isolation, Durability), which are critical when managing student grades, transcript modifications, and enrollment logs.
- **Database Architecture and Schema Design:** Engineered using **DBeaver Enterprise**. Relational schemas are normalized to the Third Normal Form (3NF) to eliminate data redundancy. Critical database optimizations applied include:
 - Composite Indexing on highly queried keys (e.g., `student_id` paired with `course_id`).
 - Foreign key constraints with strict `ON DELETE RESTRICT` configurations to prevent accidental cascading data loss of student academic histories.

2.5 Infrastructure and DevOps Layer (Isolation Deployment)

To eliminate the classic "it works on my machine" operational paradox, the entire software infrastructure is institutionalized via modern DevOps tools.

- **Containerization Stack:** Orchestrated using **Docker** and configured via `docker-compose`. The system is modularized into isolated multi-container systems:
 - `brainlyx-backend-container`: Isolating the Node.js microservice environment.
 - `brainlyx-database-container`: Hosting the isolated MySQL service instance with persistent Docker Volumes.
- **Environment Parity:** Docker configuration images enforce uniform Node.js execution runtimes, OS-level dependencies, and environment variables across local workstations and production deployment instances.

2.5.1 User Authentication Interfaces

The system provides a sleek, secure, and user-friendly interface for authentication. It features standard traditional login forms alongside single-click identity provider options to reduce onboarding friction.

- **Traditional Authentication Interface:** Includes input validation indicators, password visibility toggles, and state persistence controls.
- **Third-Party Integration Interface:** A dedicated OAuth2 interface that enables swift, one-tap access through external corporate or educational identity services.

2.6 Authentication and Security Layer

Positioned between the client application and the core data services, this layer governs identity verification, access control, and API endpoint protection.

- **Self-Managed Authentication (Local Auth):** Handled via **Node.js** and Express backend services. Passwords are securely hashed using `bcrypt` before database persistence, and session states are managed using stateless, digitally signed JSON Web Tokens (JWT).
- **Social Authentication Integration (Google OAuth 2.0):** Implements institutional Google Sign-In, verifying identity via secure token exchange with Google's authorization servers to ensure academic compliance and ease of access.

2.7 Data and Infrastructure Layer

This layer acts as the foundation of BrainlyX-LMS, responsible for business logic execution, relational data storage, schema management, and environment isolation.

- **Database Management System (DBMS):** Utilizes **MySQL**, a robust relational database engine, to structure course contents, student enrollments, grading matrixes, and analytical logs.
- **Database Administration & Modeling:** **DBeaver** Enterprise/Community utility is utilized as the primary database administration tool for schema designing, query optimization, and data migration.
- **DevOps & Containerization:** The entire application stack (Backend service, Database engine, and Cache components) is encapsulated inside isolated **Docker** containers. This guarantees structural consistency between the local development environment and production systems, minimizing environment-specific deployment bugs.